T5. Modelado Estructural Avanzado

Definición general

El **modelado estructural** en UML se refiere a la representación de la estructura **estática** del sistema, es decir, que cosas hay en el sistema y cómo están se **relacionan** entre ellas (clases, objetos, relaciones, interfaces...). Se diferencia del modelado dinámico, en que no se enfoca en el comportamiento (interacciones, eventos y flujos).

Niveles de abstracción

UML permite modelar un sistema en varios **niveles** de detalle, lo cual es crucial para diferenciar entre lo que se quiere hacer, cómo se piensa hacer y cómo se implementa finalmente.

- **Conceptual**

Es la representación del dominio de forma abstracta, sin pensar en software. Por ejemplo, en una tienda se tienen conceptos como Cliente, Producto o Pedido.

- **Especificación**

Es la definición de **interfaces** del sistema (cómo interactúan los componentes entre sí). Por ejemplo, sabemos que Cliente puede "realizarPedido()", pero no como lo hace, o que el nombre: String.

- **Implementación**

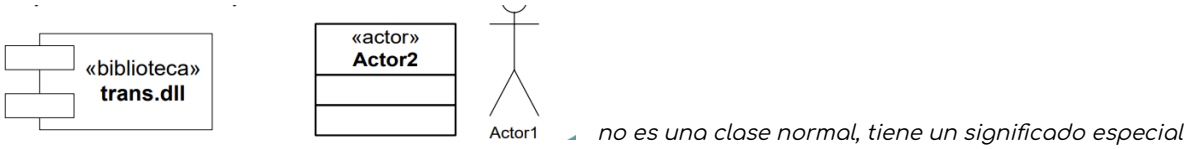
Ya se ve el detalle real en código, clases, métodos y campos. Por ejemplo, Cliente tiene "List<Pedido>" como atributo, y un método "crearPedido()".

Mecanismos de extensibilidad

Son mecanismos para ampliar UML de forma controlada, de modo que permiten adaptar el lenguaje a las necesidades específicas del dominio.

- **Estereotipos (« »)**

Extienenden el vocabulario de UML para derivar/crear **nuevos bloques** de construcción. Se aplican a bloques existentes para definir otros con características, semántica y notación propias. Se especifican entre **comillas tipográficas**. Además, UML aporta estereotipos estándar.

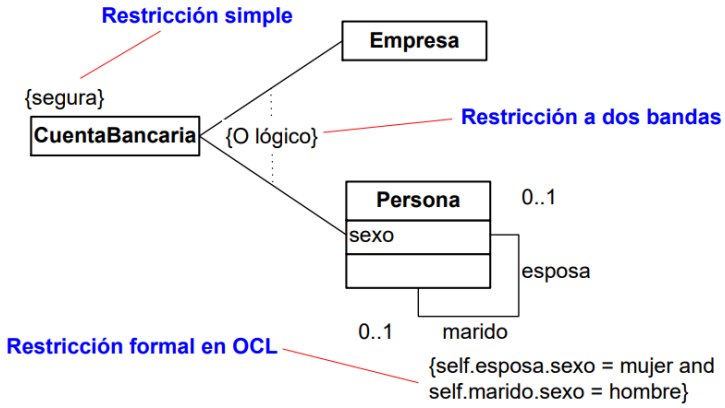


- <u>Valores etiquetados ({ })</u>

Permiten añadir **nuevas propiedades** a elementos primitivos o estereotipados. Se **aplican** al propio elemento y no a sus instancias, se representan entre llaves ({autor = "Juan"} o {versión = 2}).

- <u>Restricciones ({ })</u>

Permiten añadir **nueva semántica** o modificar reglas existentes. Especifican condiciones que aseguran modelos bien formados. Son **condiciones** que deben **cumplirse** ({ordered}, {unique} o {readOnly}).



Clasificadores

Un **clasificador** es un mecanismo que describe características estructurales y de comportamiento. Son elementos que pueden tener **instancias**: clases, interfaces, componentes, nodos, casos de uso...

Visibilidad

Controla quién puede acceder a atributos y operaciones:

- **Pública (+)**: cualquier clasificador externo, accesible desde cualquier clase.

- **Protegida (#)**: cualquier descendiente del clasificador, desde subclases.
- **Privada (-)**: sólo el propio clasificador, sólo dentro de la clase.
- **Paquete (~)**: cualquier clasificador en el mismo paquete, accesible sólo dentro del mismo paquete.

```
class Cliente {
    private String nombre;   // (-)
    protected void validar(); // (#)
    public void comprar();  // (+)
}
```

Nota: La visibilidad es pública por defecto.

Elementos del modelo estructural

- <u>Atributos</u>

Pueden especificar la visibilidad, el alcance, la multiplicidad, tipo, valor inicial y las propiedades que controlan los posibles cambios.

A **nivel conceptual,** identifican propiedades relevantes (ej: nombre de un Cliente). A **nivel de especificación**, deciden tipos y restricciones (nombre: String). A **nivel de implementación**, el atributo es un **campo real en una clase**.

Sintaxis: $[visibilidad]$ $nombre$ $[multiplicidad]$ $[:\ tipo]$ $[=\ valor\ inicial]$ $[\{propiedades\}]$

ControladorRed
-puertoConsola[2..*] : Puerto

ej: +nombre: String [1] = "Juan" {readOnly}

- <u>Operaciones</u>

A **nivel conceptual**, representan **responsabilidades** (ej: realizarPedido()). A **nivel de especificación**, se definen visibilidad, parámetros, excepciones. También se incluyen operaciones **privadas** y **protegidas**, no visibles externamente.

ej: +realizarPedido(p: Pedido): Boolean

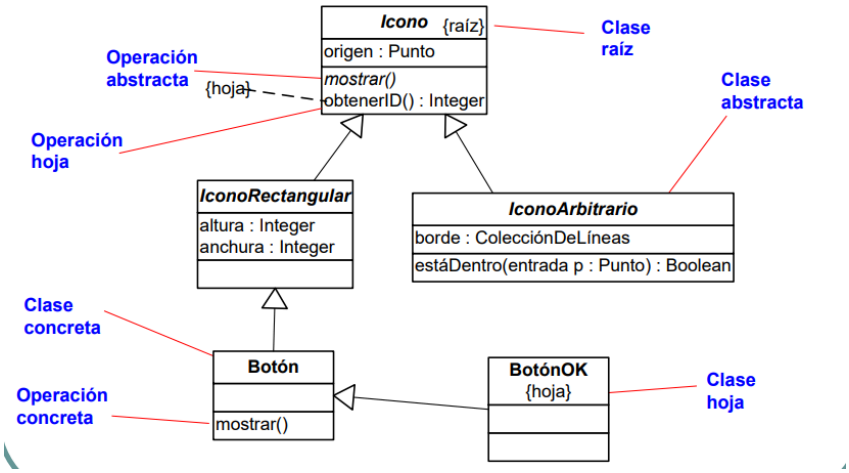
- <u>Alcance</u>

Especifica si un atributo u operación se gestiona a nivel de instancia (cada objeto tiene su copia) o si es común a todos los representantes (compartido). Existen dos tipos de alcance: de **instancia** o de **clasificador** (estático). Los atributos y operaciones con alcance de clasificador aparecen **subrayados**.

```
class Usuario {
    static int totalUsuarios; // atributo de clase (subrayado en UML)
    String nombre;            // atributo de instancia
}
    ...
```

- <u>Elementos abstractos, raíces y hojas</u>

- **Clase abstracta**: no se puede instanciar (nombre en cursiva).
- **Clase hoja ({leaf})**: no especializable, no se puede heredar.
- **Clase raíz ({root})**: no generalizable, no puede heredar de nadie.
- **Operación abstracta**: implementación diferida a las clases hijas.
- **Operación hoja**: no redefinible mediante polimorfismo por nngún hijo.



- <u>Multiplicidad</u>

Existe la **multiplicidad de clase** que es el número de instancias que puede tener.
- **Multiplicidad 0**: clase utilidad («utility»), son sólo atributos y operaciones con alcance de clasificador.
- **Multiplicidad 1**: clase unitaria («singleton»).

Existe la **multiplicidad de atributo** es el rango entre corchetes adosado al atributo.

- <u>Asociación</u>

Una **asociación es una relación de tipo** estructural. Hace uso de adornos básicos: nombre, roles, multiplicidades y agregación.

Nota: Una asociación que necesita **atributos propios** se modela como **clase asociación**.

```
# 5.4 Asociaciones y relaciones

### Asociación
Relación estructural entre clases. A distintos niveles, implica cosas distintas:

- **Conceptual**: denota **navegabilidad**: puedes ir de `Pedido` a `Cliente`.
- **Especificación**: muestra **responsabilidades** e **interfaces**.
- **Implementación**: implica una **estructura de datos** (por ejemplo, una lista)

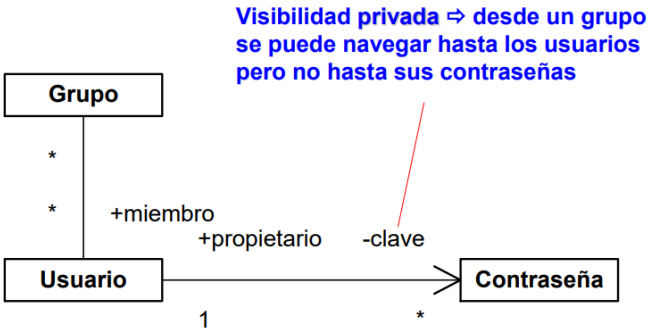
```java
class Cliente {
    List<Pedido> pedidos; // Asociación 1 a muchos
}
```
```

Tiene una serie de propiedades:

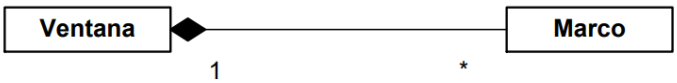
- **Navegabilidad**: Es una asociación no dirigida a priori que permite **navegar** desde los objetos de una clase a los de la otra y viceversa. En ocasiones, puede ser deseable **limitar** la navegación:

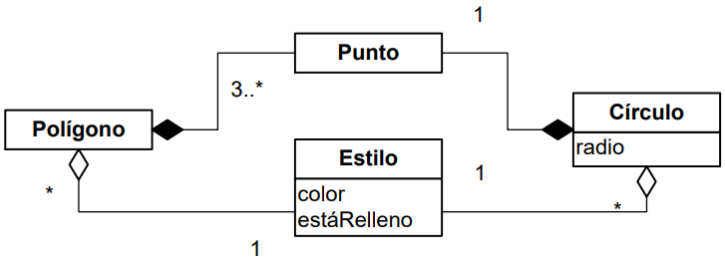
- **Visibilidad de los roles**: Puede limitarse la visibilidad de un rol desde **objetos externos** a la asociación. Existen cuatro niveles de visibilidad:

  - **Privada**: objetos del extremo no accesibles.
  - **Protegida**: objetos de ese extremo sólo accesibles a objetos hijos del otro lado.
  - **Pública**: comportamiento por defecto.
  - **Paquete**: objetos del extremo accesibles a objetos del mismo paquete.



- **Composición**: es una agregación con fuerte relación de pertenencia entre el *todo* y sus partes. Las partes se pueden crear después o eliminar antes, pero viven y mueren con el *todo*, este es responsable de gestionar **creación** y **destrucción** de las partes. Un objeto sólo puede formar parte de **un todo**.





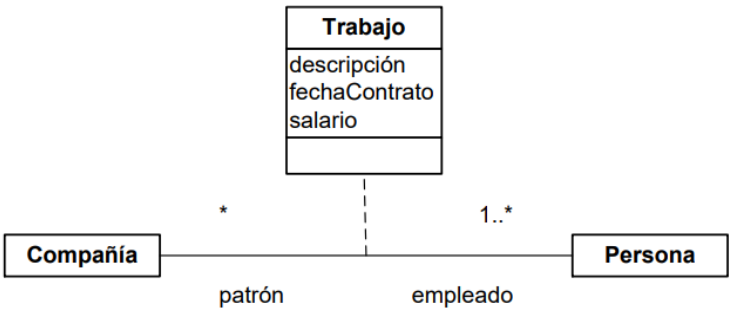
Nota: Composición vs Agregación

- **Agregación**: relación débil. Las partes pueden vivir sin el todo.
- **Composición**: relación fuerte. Las partes viven y mueren con el todo.

```
class Pedido {
    private List<LineaPedido> lineas; // Composición: LineaPedido no existe sin
}
```

Clase asociación

La clase asociación es un elemento que aglutina propiedades de asociaciones y clases.



Interfaces y realización

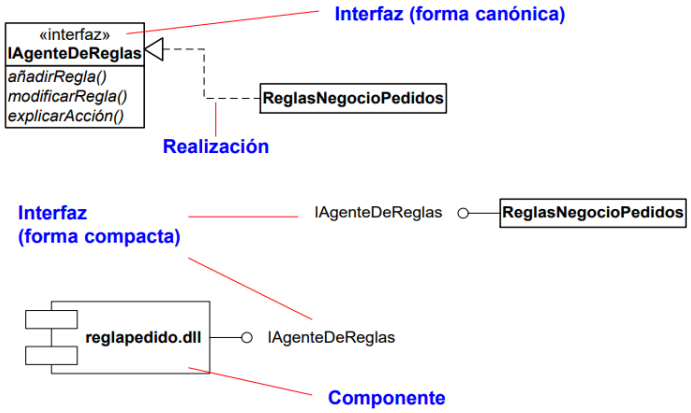
Una **interfaz** es una **colección** de **operaciones** que especifica un *servicio* prestado por una **clase** o un componente. Es una *frontera* entre el comportamiento deseado para una abstracción (¿qué hace?) y su implementación (¿cómo lo hace?). **No aportan estructura** (no incluyen atributos) ni **implementación** (no proporcionan métodos).

Una interfaz puede **implicarse** en **relaciones** de generalización, asociación, dependencia y realización. Especifica un contrato para una clase o un componente sin dictar su implementación (no atributos ni implementación, sólo define métodos).

La **realización** es la relación semántica entre dos clasificadores en la que uno disfruta de un servicio garantizado por el otro.

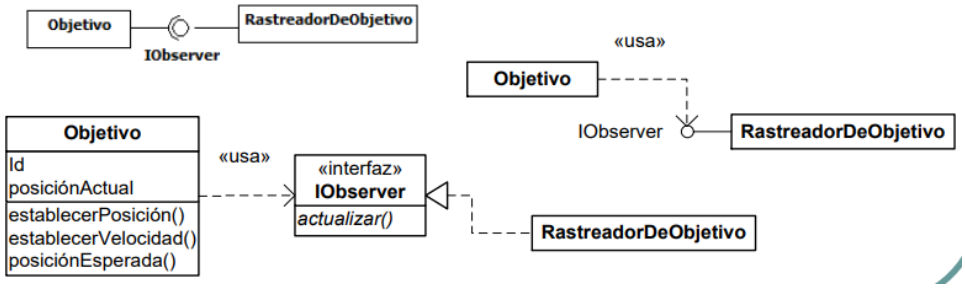
```
interface Imprimible {
    void imprimir();
}

class Factura implements Imprimible {
    public void imprimir() {
        // implementación concreta
    }
}
```



- <u>Implementación y uso de una interfaz</u>

Una clase o un componente puede **realizar muchas** interfaces (las provee). También puede **depender** de una o varias interfaces ofrecidas por otros (las consume).



- <u>Interfaz bien estructurada</u>

Debe ser **sencilla** y **completa,** de modo que proporcione todas las operaciones necesarias y suficientes para un solo servicio. Debe ser **comprensible,** que proporcione suficiente información para permitir su uso y su realización sin examinar los usos o las implementaciones existentes. Finalmente, tiene que ser **manejable**, que guíe hacia las propiedades clave sin saturar con los detalles.